

Lab 01: The Rental Refund Crisis

Name: Nguyen Ho Ngoc Minh ID: 11247201 DS66B

Choose rental_id = 2, customer_id = 459

Query	Query History
<pre>1 SELECT rental_id, customer_id FROM rental LIMIT 1;</pre>	
Data Output	Messages Notifications
Showing rows: 1 to 1	
rental_id [PK] integer	customer_id smallint
1	2 459

Initial State:

Query Query History 

```
1 SELECT rental_id, customer_id, return_date
2 FROM rental
3 WHERE rental_id = 2;
4
```

Data Output Messages Notifications



Showing rows: 1 to 1   Page 1

	rental_id [PK] integer 	customer_id smallint 	return_date timestamp without time zone 
1	2	459	2005-05-28 19:40:33

Exercise 1:

Fault-Injected Transaction Execution

```
16 SELECT return_date
17 FROM rental
18 WHERE rental_id = 2;
```

Data Output Messages Notifications

ERROR: current transaction is aborted, commands ignored until end of transaction block

SQL state: 25P02

Query Query History

```
5 BEGIN;
6
7 UPDATE rental
8 SET return_date = NOW()
9 WHERE rental_id = 2;
10
11 INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
12 VALUES (459, 1, 2, 99.99, 'WRONG_DATE_FORMAT');
13
14 COMMIT;
```

Data Output Messages Notifications

ERROR: invalid input syntax for type timestamp: "WRONG_DATE_FORMAT"
LINE 8: VALUES (459, 1, 2, 99.99, 'WRONG_DATE_FORMAT');
A

SQL state: 22007
Character: 174

Post-execution State Verification

```
16 SELECT return_date
17 FROM rental
18 WHERE rental_id = 2;
```

Data Output Messages Notifications

Showing rows: 1 to 1

	return_date timestamp without time zone
1	2005-05-28 19:40:33

Is the return_date updated? Why not?

No. The return_date attribute for rental_id = 2 remains identical to its baseline value prior to the transaction execution.

When the INSERT statement in Step 2 encountered an unparseable timestamp literal (SQL state: 22007), PostgreSQL immediately transitioned the transaction into an **Aborted State**.

Once a transaction is aborted, subsequent operational commands—including the COMMIT instruction—are discarded. PostgreSQL automatically issues an internal **ROLLBACK**.

The prior UPDATE operation, which was only applied to transient in-memory transaction buffers, was completely discarded. This guarantees the **Atomicity** rule: either every operation within the boundary succeeds and commits, or the database state remains untouched.

Excercise 2: The “Flexible” Staff Task

Automated PL/pgSQL Implementation

Query

Query History

Scratch Pad X

```
19 DO $$
20 DECLARE
21     new_staff_id INT;
22 BEGIN
23     -- Step 1: Register a new staff member with a temporary valid store_id
24     INSERT INTO staff (first_name, last_name, address_id, email, store_id, active, username, password)
25     VALUES ('Minh', 'Nguyen', 1, 'minh.nguyen@sakilastaff.com', 1, true, 'minhnguyen', 'securepass12
26     RETURNING staff_id INTO new_staff_id;
27     -- Steps 2 & 3: Attempt store reassignment within a sub-transaction block (Savepoint equivalent)
28     BEGIN
29         -- Intentional foreign key constraint violation (store_id = 9999 does not exist)
30         UPDATE staff
31         SET store_id = 9999
32         WHERE staff_id = new_staff_id;
33     EXCEPTION
34         WHEN foreign_key_violation THEN
35             -- Step 4: Catch error, perform rollback, and reassign to valid Store 1
36             RAISE NOTICE 'Store 9999 not found! Rolling back to savepoint and assigning to Store 1..
37             UPDATE staff
38             SET store_id = 1
39             WHERE staff_id = new_staff_id;
40     END;
41     -- Step 5: Entire transaction automatically commits successfully upon exit
```

Data Output

Messages

Notifications

NOTICE: Staff member created successfully with staff_id: 3
DO

Query returned successfully in 156 msec.

Total rows: Query complete 00:00:00.156 CRLF Ln 18, Col 21

Pure Standard SQL Implementation

Query

Query History

Scratch Pad X

```
45 BEGIN;
46 -- Step 1: Register a new staff member
47 INSERT INTO staff (first_name, last_name, address_id, email, store_id, active, username, password)
48 VALUES ('Minh', 'Nguyen', 1, 'minh.nguyen@sakilastaff.com', 1, true, 'minhnguyen', 'securepass123');
49 -- Step 2: Establish a checkpoint marker
50 SAVEPOINT staff_registered;
51 -- Step 3: Attempt assignment to a non-existent store (triggers foreign key error)
52 UPDATE staff
53 SET store_id = 9999
54 WHERE username = 'minhnguyen';
55 -- Step 4: Revert only the failed assignment back to the savepoint
56 ROLLBACK TO staff_registered;
57 -- Reassign the staff member to valid Store 1
58 UPDATE staff
59 SET store_id = 1
60 WHERE username = 'minhnguyen';
61 -- Step 5: Commit all valid operations to the database
62 COMMIT;
```

Data Output

Messages

Notifications

COMMIT

Query returned successfully in 87 msec.

Total rows: Query complete 00:00:00.087 CRLF Ln 62, Col 8

✓ Query returned successfully in 87 msec. ✕

Verification Query

```

64  -- Verify that the staff member was created and assigned to store_id = 1
65  SELECT staff_id, first_name, last_name, email, store_id, active, username
66  FROM staff
67  WHERE username = 'minhnguyen';

```

staff_id	first_name	last_name	email	store_id	active	username
3	Minh	Nguyen	minh.nguyen@sakilastaff.c...	1	true	minhnguyen
4	Minh	Nguyen	minh.nguyen@sakilastaff.c...	1	true	minhnguyen

Total rows: 2 Query complete 00:00:00.101 CRLF Ln 63, Col 1

When attempting to update **store_id** to **9999**, the database engine triggers a **foreign_key_violation** exception. By issuing a **ROLLBACK TO <savepoint_name>**, only the faulty **UPDATE** is undone, leaving the initial staff registration intact in the transaction buffer. The subsequent reassignment to **store_id = 1** succeeds, and **COMMIT** finalizes the creation of the staff member.